Loot Economy Simulator — Design Document

Section 1 — Project Overview

The Loot Economy Simulator is a probability and statistics application modeled after video game loot table rarity systems. The project allows users to build item-based loot tables, configure probability behavior, run repeatable simulations, and analyze the statistical outcomes of randomized reward systems.

The simulator is based on the idea that many games use weighted randomness to determine which items players receive from loot boxes, chests, packs, drops, or reward pools. Instead of treating this as only a game mechanic, this project uses loot tables as a practical way to explore probability, randomness, expected outcomes, duplicate behavior, value distribution, and simulation-based statistical analysis.

The project is designed as a full-stack statistical visualization platform with game-inspired aesthetics. The goal is not simply to simulate random rewards, but to help users understand how probabilistic systems behave over repeated simulations.

The application will support deterministic seeded randomness, configurable loot systems, duplicate handling systems, simulation comparison, and chart-ready statistical outputs.

Section 2 — Core Problem / Purpose

The Loot Economy Simulator is designed as a probability visualization and simulation platform modeled after video game loot systems. While the project uses game-inspired aesthetics and mechanics such as item rarities, duplicate rewards, and currency compensation systems, the core purpose of the application is educational and analytical rather than purely game-focused.

The primary purpose of the project is to help users understand how randomness, probability distributions, averaging, convergence, and statistical variance behave across repeated simulations. The application is intended to make abstract statistical concepts easier to understand by allowing users to interact with a visual and game-like probability system.

The project is designed for multiple types of users, including:

- statistics students learning probability and simulation concepts
- developers interested in Monte Carlo simulation and seeded randomness
- game designers experimenting with loot table balancing
- general users interested in understanding randomized reward systems

Although the system supports several use cases, the primary target audience is users interested in statistics and probability visualization.

One of the primary questions the application attempts to answer is:

> “How different are the actual simulated results from the listed probabilities, and how fair does the system feel statistically?”

A secondary focus of the application is duplicate reward analysis. Users can investigate how frequently duplicate items occur and how much duplicate compensation currency is generated under different loot table configurations.

The project combines several perspectives:

- a statistics learning tool
- a game economy experimentation tool
- a full-stack simulation platform

However, the primary framing of the project is educational and statistical.

Section 3 — Target User Experience

The Loot Economy Simulator is intended to feel like a professional statistical analysis dashboard with video game-inspired aesthetics layered on top.

The visual design should prioritize:

- clarity
- responsiveness
- chart readability
- fast experimentation
- minimalistic UI structure

The application will support:

- light mode
- dark mode
- customizable primary colors
- customizable secondary colors

The primary user flow is:

Create Loot Table ↓ Configure Simulation Settings ↓ Run Simulations ↓ Analyze Statistical Results ↓ Compare Simulation Configurations ↓ Save Simulations

The application should encourage rapid experimentation and iterative statistical exploration.

Simulation comparison in Version 1 will support side-by-side analysis using:

- charts
- distribution tables

- statistical panels
- duplicate metrics
- currency metrics

Primary user experience priorities: 1. Fast simulation performance 2. Statistical clarity 3. Extensible architecture

---

Section 4 — Core Features

Loot Table Structure

The loot system uses a two-stage probability model.

Stage 1:

- select a rarity

Stage 2:

- select an item from the chosen rarity group

The loot table is divided into two sections.

Rarity Section

Each rarity contains:

- rarity label
- rarity color
- rarity probability OR weight

Rarity labels are fully customizable and cosmetic.

Example:

- Common
- Rare
- Epic
- Legendary
- Mythic

The rarity system defines the first-stage probability roll.

Item Section

Each item contains:

- item name
- assigned rarity
- optional item value

Items inherit their probability from the rarity they belong to.

Item Selection Modes

The system supports two item selection modes.

Equal Chance Mode

Once a rarity is selected, all items within that rarity have equal probability.

Example:

Rare rarity selected.

Rare items:

- Iron Sword
- Magic Ring
- Silver Bow

Each item has equal chance.

Weighted Item Mode

Users may optionally assign item-level probabilities or weights within a rarity.

Example:

Rare rarity = 25%

Rare items:

- Iron Sword = 30%
- Magic Ring = 50%
- Silver Bow = 20%

Final probability:

- Iron Sword = 7.5%
- Magic Ring = 12.5%
- Silver Bow = 5%

This allows advanced users to create more granular probability systems.

Probability Configuration

The application supports:

- percentage mode
- weight mode

In weight mode, values are normalized automatically and displayed as percentages.

Simulation Configuration

Users can configure:

- pulls per simulation
- simulation count
- deterministic seed
- probability mode
- duplicate handling mode
- item weighting mode

Total pulls:

pulls per simulation × simulation count

Duplicate Handling Modes

Hard Duplicate Prevention

Once an item has been obtained during a simulation, it cannot appear again.

Maximum pulls are limited to the number of unique items.

Duplicate Currency Compensation

Duplicates remain possible.

Duplicate pulls generate one global compensation currency.

Lower probability items generate larger compensation rewards.

Currency mappings are generated programmatically using item probability data.

Seeded Randomness

The simulation system supports deterministic seeded randomness.

Identical:

- seeds
- loot tables
- simulation settings

must produce identical outputs.

Seeds are:

- optional
- auto-generated if omitted
- reusable
- savable

Saved Simulations

Users can save up to 10 simulation configurations locally.

Saved simulations contain enough information to reproduce identical results.

Initial persistence uses local storage with future database migration support.

Statistical Outputs

Version 1 statistical outputs may include:

- average value earned
- average duplicates
- total currency earned
- rarity frequencies
- observed vs expected probabilities
- variance

- standard deviation
- confidence intervals
- longest unlucky streak
- pulls until first rare item
- pulls until collection completion

The frontend primarily displays:

- histograms
- line graphs
- distribution tables
- bar charts
- pie charts
- chart-ready statistical datasets

Core Statistical Visualizations

Version 1 visualizations include:

Rarity Frequency Bar Chart

```
X-axis = rarity
Y-axis = pull count
```

Used to visualize rarity distribution frequencies across simulations.

Item Frequency Bar Chart

```
X-axis = item
Y-axis = pull count
```

Used to visualize item-level distribution frequencies.

Rarity Percentage Pie Chart

Displays rarity percentages as portions of total pulls.

Item Percentage Pie Chart

Displays item percentages as portions of total pulls.

Pull Distribution Histogram

Used to visualize:

- pulls until rare item
- pulls until legendary item
- pulls until collection completion

Convergence Line Graph

Displays how observed probabilities stabilize over increasing pull counts.

Example:

```
5 pulls → 20%
100 pulls → 7%
10000 pulls → 5.01%
```

This graph demonstrates:

- convergence behavior
- randomness stabilization
- law of large numbers

Sampling Distribution Histogram

The application supports visualization of sampling distributions.

Example process:

- run many simulations
- calculate average values from each simulation
- graph the distribution of those averages

This visualization demonstrates:

- averaging behavior
- central limit theorem concepts
- bell curve formation
- distribution of sample means

Backend Architecture

Frontend Stack

The frontend stack includes:

- React
- TypeScript
- Tailwind CSS
- Recharts

Backend Stack

Architecture:

```
React Frontend
↓
Express API Gateway
↓
FastAPI Simulation Service
```

Express acts as the frontend-facing API gateway.

FastAPI acts as the dedicated simulation and statistical computation service.

Express Responsibilities

Express responsibilities include:

- request validation
- frontend API routing
- payload transformation
- coordinating requests to the Python simulation service

Version 1 primarily uses Express as a lightweight proxy/orchestration layer.

FastAPI Responsibilities

FastAPI responsibilities include:

- running simulations
- probability calculations

- duplicate handling
- seeded random generation
- statistical calculations
- chart-ready data generation

The FastAPI service is intentionally focused on simulation and statistical computation rather than persistence.

Persistence Strategy

Version 1 uses local storage for persistence.

Future persistence systems should support:

- PostgreSQL
- cloud synchronization
- saved accounts
- simulation sharing

The architecture should remain extensible for future database-backed persistence.

Simulation Engine Constraints

Version 1 limits:

```
Maximum pulls per simulation: 1000
Maximum simulation count: 100000
```

Version 1 simulations execute synchronously.

Future versions may support:

- background simulation execution
- simulation progress tracking
- repeated server status updates
- parallel simulation execution across CPU cores

Validation Rules

Rarity Constraints

Maximum rarity count:

```
10 rarities
```

If percentage mode is selected:

- rarity probabilities must total 100%

Item Constraints

The system should support hundreds of items.

If item-level percentage mode is enabled within a rarity:

- item probabilities within that rarity must total 100%

Invalid Configurations

The frontend should block invalid simulations from executing.

Examples:

- invalid percentages
- missing required fields
- impossible pull counts
- invalid duplicate prevention settings
- empty rarity groups

Export Support

Version 1 export support includes:

- JSON exports
- CSV exports

Future export support may include:

- PNG chart exports
- PDF reports
- advanced statistical summaries

Future Features

Potential future features include:

- pity systems
- multiplayer/shared loot tables
- cloud accounts
- public seed sharing
- advanced statistical insights
- CSV/JSON loot table importing
- mock data generation tools
- background simulation processing
- simulation progress tracking
- parallel simulation execution

CSV and JSON loot table importing is expected to be one of the first major post-Version-1 features.

Raw Pull Log Strategy

The application returns raw pull-by-pull logs only when doing so remains useful and manageable.

Full Raw Logs

If the total number of generated pull events is less than or equal to 1000, the backend may return the complete raw simulation log.

```
total pull events = simulation count × pulls per simulation
```

Example:

```
5 simulations × 20 pulls = 100 total pull events
```

In this case, all simulation logs can be returned.

Partial Raw Logs

If total pull events exceed 1000, the backend should still return:

- aggregated statistics
- chart-ready datasets
- summary metrics

For raw logs, it should only return enough complete simulations to stay within the 1000-pull preview limit.

Example:

```
100 simulations × 50 pulls = 5000 total pull events
```

The response may include the first 20 complete simulations:

```
20 simulations × 50 pulls = 1000 preview pull events
```

This avoids fragmented random pulls and preserves simulation continuity.

Saved Simulation Strategy

Saved simulations store:

- loot table configuration
- rarity configuration
- item configuration
- simulation settings
- seed
- duplicate mode
- chart-ready datasets
- calculated statistics
- raw logs only if they are within the returned pull-log limit

Saved simulations should avoid storing massive raw datasets. Large simulations should persist aggregated results and chart-ready data instead.

Statistics Scope

Statistics are organized into three levels:

Per-Item Statistics

Examples:

- expected probability
- observed probability
- probability difference/error
- pull count
- duplicate count

- currency generated
- average pull position
- first appearance pull average

Per-Rarity Statistics

Examples:

- expected rarity probability
- observed rarity probability
- rarity frequency
- total rarity currency
- average pulls until rarity
- rarity completion rate

Global Simulation Statistics

Examples:

- mean total value
- median total value
- standard deviation
- confidence intervals
- average duplicates
- average collection completion
- total pulls

The system should avoid attaching every statistic to every entity. Item-level, rarity-level, and global statistics should remain clearly separated.

V1 Visualization Set

Version 1 should include a focused graph set rather than every possible visualization.

Shared Graphs

These graphs apply to both normal duplicate mode and hard prevention mode.

1. Rarity Frequency Bar Chart

- Graph type: bar chart
- Purpose: compare rarity pull counts
- X-axis: rarity labels

- Y-axis: pull count
- Required data: rarity label, pull count, observed percentage
- Optional mode: currency generated per rarity

---

2. Item Frequency Bar Chart

- Graph type: bar chart
- Purpose: compare item pull counts
- X-axis: item names
- Y-axis: pull count
- Required data: item name, rarity, pull count, observed percentage
- Optional mode: currency generated per item

---

3. Rarity Distribution Pie Chart

- Graph type: pie chart
- Purpose: show each rarity as a share of total pulls
- Slice value: rarity pull count or rarity percentage
- Required data: rarity label, pull count, observed percentage
- Optional mode: rarity share of total duplicate currency

---

4. Item Distribution Pie Chart

- Graph type: pie chart
- Purpose: show each item as a share of total pulls
- Slice value: item pull count or item percentage
- Required data: item name, pull count, observed percentage
- Optional mode: item share of total duplicate currency

---

5. Convergence Line Graph

- Graph type: line graph
- Purpose: show observed probability stabilizing over time
- X-axis: pull number or cumulative pull bucket
- Y-axis: observed probability percentage
- Required data: cumulative pull count, observed rarity/item percentage at each checkpoint, expected probability line
- Statistical concept: law of large numbers

---

6. Sampling Distribution Histogram

- Graph type: histogram

- Purpose: show distribution of averages across repeated simulations
- X-axis: average value or average reward metric per simulation
- Y-axis: simulation count/frequency
- Required data: per-simulation averages, grouped bins, frequency per bin
- Statistical concept: central limit theorem and distribution of sample means

---

Hard Prevention-Specific Graphs

Hard prevention mode guarantees eventual collection if pull count equals the number of unique items. Because final item frequency becomes deterministic, hard prevention mode should focus on acquisition order, collection progression, and timing.

7. Pull Position Distribution Histogram

- Graph type: histogram
- Purpose: show when a target rarity or item tends to appear
- X-axis: pull position
- Y-axis: frequency
- Required data: first appearance pull position for selected item or rarity across simulations
- Applies to: hard prevention mode

---

8. Collection Progression Curve

- Graph type: line graph
- Purpose: show collection completion over time
- X-axis: pull number
- Y-axis: average collection completion percentage
- Required data: pull index, unique items collected by each pull, total unique item count, average completion percentage across simulations
- Applies to: hard prevention mode

This graph shows how quickly users complete the collection. Early pulls may increase completion quickly, while later pulls may slow down as the remaining uncollected items become rarer. This gives hard prevention mode a meaningful statistical purpose beyond simply producing the same final collection in different orders.

---

9. Average Pulls Until Milestone

- Graph/table type: summary table or compact bar chart
- Purpose: show acquisition timing milestones
- Possible milestones: first rare, first legendary, 50% collection, 75% collection, full collection
- Required data: milestone name, average pull count, median pull count, standard deviation
- Applies to: hard prevention mode

Comparison Behavior

Version 1 supports comparison between:

- a newly generated unsaved simulation and a saved simulation
- two previously saved simulations

Comparison views should be side-by-side by default and include delta statistics.

Example:

```
Legendary Rate
Run A: 4.9%
Run B: 6.2%
Difference: +1.3%
```

Delta statistics should help users quickly identify how two configurations differ.

Monte Carlo Wording

The UI should use approachable wording such as:

```
Run Simulation
```

The term Monte Carlo simulation should appear in:

- documentation
- help text
- educational tooltips
- project explanation sections

This keeps the interface approachable while still connecting the project to formal statistical terminology.

V1 Cutoff

The following features are intentionally excluded from Version 1:

- pity systems
- multiplayer/shared loot tables
- cloud accounts
- background simulation jobs
- progress tracking

- parallel execution
- advanced exports such as PNG/PDF reports
- CSV/JSON loot table importing
- item images/assets

These features should be treated as post-Version-1 roadmap items.

Portfolio / Career Positioning

The project is positioned as both:

- a statistical simulation platform
- and a game-inspired loot economy simulator

Primary portfolio focus:

- full-stack architecture
- probabilistic simulation systems
- seeded deterministic randomness
- statistical visualization
- data-driven experimentation

One of the most technically interesting features of the project is deterministic seeded simulation reproducibility.